SkillSwap interview prep

Everything here is taken from the code, `CHANGELOG.md` and `README.md` in this repository. Answers are written the way you would say them out loud. Links point at the file that proves each answer.

Contents

0. Read this first
1. The pitch
2. Numbers to know, and where each one comes from
3. Architecture and the path of one exchange
4. Technical questions by area
5. Behavioural questions and stories
6. The awkward questions
7. System design follow-ups
8. Concepts to explain from scratch
9. Before the interview

---

0. Read this first

- **SkillSwap is not on your resume** (`Portfolio/public/resume.pdf`), but it is on your portfolio and GitHub. It is arguably your strongest engineering project (tests, CI, WebRTC, CRDTs, security). Add it.
- **Your resume's SynChat line ("JWT in HTTP-only cookies, bcrypt, MongoDB/Mongoose") actually describes SkillSwap**, not today's SynChat. If an interviewer asks about "the JWT chat app", be ready to say which project does what.
- **`CLAUDE.md` is committed to this repository.** It is a conventions file for "people and coding agents". An interviewer may ask whether an AI wrote the code. Answer honestly about the tools you used, then show ownership by explaining any part in depth.
- **Version 2.0 is a rebuild of a 1.0 prototype.** The changelog lists what was wrong with 1.0 (hard-coded JWT secret, token in `localStorage`, invented ratings, a fake code runner). That makes great interview material (section 5), as long as you can explain each fix.

---

1. The pitch

One sentence

SkillSwap is a peer-to-peer skill exchange: you list what you can teach and what you want to learn, get matched with someone whose skills and schedule fit, agree a plan, and meet in a live learning room with video, a shared code editor, a whiteboard and notes. Nobody pays; each side teaches the other.

Thirty seconds

Swapping skills needs three things that rarely come together: a partner who fits, an agreement on what each side teaches, and a place to actually do it. SkillSwap ranks members with an explainable match score, lets two people negotiate an exchange (accept, decline or counter), turns an accepted exchange into a contract with a number of sessions, and gives each session a room: a peer-to-peer WebRTC call with screen sharing, plus a code editor, notes and a whiteboard that sync live through a CRDT and are saved automatically. JavaScript typed in the editor runs in a locked-down sandbox. Reputation only counts sessions both people actually attended. It is a TypeScript monorepo, React 19 and Express 5 on MongoDB with Socket.IO, with 224 unit and integration tests, Playwright end-to-end tests that hold a real two-browser call, and WCAG 2.2 AA checks in CI.

Two minutes: add these, in order

1. **It is a rebuild.** Version 1.0 was a prototype with real security bugs and invented numbers. 2.0 fixed those and added tests; the changelog lists every fix.
2. **The shared contract.** Every request body is one Zod schema in `shared/`, used by the server to validate and by the client's forms. Every response goes through one serializer module, so a private field cannot leak by accident.
3. **Correctness under concurrency without transactions.** One open exchange per pair is a unique index; every state change is a conditional update that only matches the state it validated; ratings are recomputed from reviews, never incremented.
4. **The learning room** is the technically interesting part: WebRTC "perfect negotiation", Yjs for conflict-free shared editing with server persistence and offline catch-up, and a code sandbox with an opaque origin, no network and a hard timeout.
5. **Honest limits.** One process holds room state in memory, calls across strict NATs need a TURN server, and the free host loses uploaded files on restart.

What is the hard part?

The learning room: getting a two-person call, three collaborative editors and a code runner to work reliably across reconnects, late joiners, second tabs, cameras that appear late and screens being shared, without either user losing an edit or the call needing a "caller".

---

2. Numbers to know, and where each one comes from

What	Number	Source / why
Unit and integration tests	**224**: 13 shared, 123 server, 88 client	`npm test` (run it yourself and quote what it prints). The README says "200+".
End-to-end tests	27 per the changelog (6 spec files: auth, exchange flow, admin, public pages, responsive, sandbox)	`npm run build && npm run e2e`

What	Number	Source / why
Match score	out of 100: skills 50, availability 20, level 10, language 10, preferences 10	`MATCH_WEIGHTS` in constants.ts
Discover candidates ranked per request	at most 500	`MAX_CANDIDATES` in users.ts
Room code	`SWAP-XXXX-XXXX-XXXX`, 12 characters from a 32-letter alphabet ≈ 60 bits	ids.ts
Session cookie lifetime	7 days (`SESSION_DAYS`)	env.ts
bcrypt cost	12 (4 in tests to keep them fast)	env.ts
Password	8 characters minimum, 72 bytes maximum	`LIMITS` in constants.ts
HTTP rate limits (production)	sign-in/sign-up 20 per 15 min per IP; API 300 per min; uploads 30 per 10 min	security.ts
Socket rate limits (token buckets)	chat 8 per 4 s; document updates 120/s; call signals 200/s	realtime/index.ts
Shared document	256 KB max per update, 2 MB max stored, saved 2 s after the last change	rooms.ts, constants.ts
Code sandbox	5 s timeout, 200 output lines, 2000 characters a line	runSandbox.ts
Shared files	10 MB each (`MAX_UPLOAD_MB`), 20 per session, extension allow-list	sessions.ts
Chat history on join	last 100 messages, 2000 characters each	realtime/index.ts
JSON body limit	100 KB	app.ts
Demo data reset	every 24 hours on the public demo	render.yaml

There are no performance benchmarks in the repository. If asked "how fast is it", say what you did for speed (precompressed brotli assets, immutable caching of hashed files, code-split pages, font preload) and offer to measure.

---

3. Architecture and the path of one exchange

```
shared/   zod schemas, constants, TypeScript types (used by both sides)
server/   Express 5 API, Socket.IO, Yjs rooms, Mongoose models; serves the built client
client/   React 19 SPA (Vite 8, Tailwind 4, TanStack Query, React Router 7, CodeMirror 6)
e2e/      Playwright tests against the built app

Browser --HTTP /api------> Express: routes -> services -> Mongoose -> MongoDB
   |--WebSocket /socket.io--> Socket.IO: presence, notifications, room seats, chat,
   |                          call signalling relay, Yjs document merge + persistence
   '--WebRTC (UDP)----------> the other participant's browser (audio, video, screen)
                              (STUN to discover addresses; TURN relay if configured)
```

One Node process serves everything in production. In development Vite serves the client and proxies `/api` and `/socket.io`.

Server layering (server/src): `routes/` parse input with the shared schemas and call `services/`; services hold every rule (who may do what, state transitions, notifications); `models/` are Mongoose schemas; serializers.ts is the only place a database document becomes an API response.

The path of one exchange, out loud

1. **Sign up and onboard.** Add at least one skill to teach and one to learn, your availability (days and parts of the day), languages and session preferences.
2. **Discover.** Members are filtered in MongoDB, scored against you in Node, sorted by match, then paginated. Open any score to see its five parts and the reasons ("Two-way swap: you teach Java, they teach Video Editing").
3. **Propose.** Pick what you teach and what you want, a number of sessions, a duration and goals. The server checks you actually list the skill you offer and they list the one you want, and a unique index guarantees one open exchange per pair. The other member gets a live notification.
4. **Negotiate.** The person who did not make the latest offer can accept, decline or counter (change sessions, length, schedule). The one who made it can withdraw. Every step is a conditional update.
5. **Accept → contract.** Accepting creates a learning contract with the agreed terms and goals.
6. **Schedule.** Either side schedules sessions against the contract, up to the agreed number. Each gets an unguessable room code.
7. **Meet.** Both open the room. The call connects peer to peer; code, notes and whiteboard sync live; chat and files are shared.
8. **End and review.** Ending a session marks it completed only if both people entered the room. Each can leave one review, and both members' ratings are recomputed from stored reviews.

---

4. Technical questions by area

4.1 Authentication: what happens to a password, and how the token works

Sign-up, step by step.

1. The form (AuthPages.tsx) uses react-hook-form with the shared `registerSchema`, so the browser shows the same rules the server enforces.
2. `fetch('/api/auth/register', { method: 'POST', body: JSON.stringify({ name, username, email, password }) })`. **The password travels in plain text in the JSON body, protected by HTTPS.** Never in a URL, never stored in the browser.
3. The server's `originGuard` checks the `Origin` header and `Sec-Fetch-Site` (see 4.2), the auth rate limiter counts the attempt, and `parse(registerSchema, req.body)` validates: username 3–24 characters, password at least 8 characters and **at most 72 bytes**, because bcrypt ignores everything after 72 bytes and silent truncation would be a nasty surprise.
4. services/auth.ts checks for an existing email or username to give a field-specific message, then `bcrypt.hash(password, 12)`. bcrypt makes a random 16-byte salt and stores `$2b$12$<salt><hash>`, so the salt and cost travel with the hash.
5. If two sign-ups race for the same email, the unique index decides (Mongo error 11000 becomes a 409). The check-then-insert alone would not be safe.
6. The `password` field is `select: false` in the schema, so no query returns it unless it explicitly asks with `+password`.
7. A session cookie is set and the response is the user as the serializer shapes it (no password, no internal fields).

Login. Find by email or username. Always run exactly one `bcrypt.compare`: against the real hash, or against a **decoy hash** when the account does not exist, so the response time does not reveal which usernames exist. One error message for both cases. Suspended accounts get a 403. Rate limit: 20 attempts per 15 minutes per IP in production.

What is in the cookie? A **JWT** signed with HS256 using `JWT_SECRET`, with two claims: `sub` (the user id) and `tv` (the user's `tokenVersion` at issue time), expiring after `SESSION_DAYS` (7). See session-token.ts. The cookie is:

- named `__Host-skillswap` in production. The `__Host-` prefix makes the browser refuse the cookie unless it is `Secure`, has `Path=/` and has no `Domain`, so it is bound to this exact host and cannot be set by a subdomain.
- `HttpOnly`: JavaScript cannot read it, so an XSS cannot steal it. That is the main fix over 1.0, which kept the token in `localStorage`.
- `SameSite=Lax`: not sent on cross-site POSTs.
- `Secure` in production.

How is each request authenticated? middleware/auth.ts: verify the signature and expiry (algorithm pinned to HS256, so a token claiming `alg: none` or RS256 is rejected), check the claims' shapes, load the user, and **reject if `user.tokenVersion !== claims.tv`**, or if the account is suspended. The user document is attached to `req.user`.

So how do you revoke a JWT? Through `tokenVersion`. Changing your password or being suspended increments it, which instantly invalidates **every** cookie issued before, on every device, and the server also disconnects that user's sockets. The device that changed the password gets a fresh cookie.

Be precise about the limitation: logout only clears the cookie in that browser. The token itself stays valid until it expires if someone had copied it. Revoking one device without signing out the others would need a per-session id (`jti`) stored server-side, at which point it is a session table. Volunteer this; it shows you understand the JWT trade-off.

Why a JWT at all, if you load the user on every request anyway? Honest answer: the user lookup is needed regardless (for role, status and the token version), so the JWT's "stateless" benefit is small here. What it gives is a self-contained, tamper-proof cookie with an expiry, and "sign out everywhere" through one integer without a sessions collection. If I needed per-device sign-out, I would switch to opaque session ids in a collection (what my SynChat project does).

What if `JWT_SECRET` is missing or weak? Production refuses to start: it must be at least 32 characters and not a placeholder. In development an ephemeral random secret is generated each run (so restarting the server signs everyone out, which is fine locally). 1.0 had a hard-coded fallback secret, meaning anyone could forge a token for any user; that was the first fix.

`/api/auth/me` returns 200 with `user: null` when signed out. Why not 401? "Who am I?" is a question every visitor's page asks; a 401 would log an error in every anonymous visitor's console. Protected endpoints do return 401, and the client's API wrapper calls a handler on 401 that clears the cached user.

Demo sign-in. One-click sign-in exists only when `DEMO_MODE` is on, and only for accounts the seed flagged `isDemo`, so it can never open a real member's account. Demo accounts cannot change their shared password, otherwise one visitor could lock everyone else out.

Is there Google sign-in? No. Email or username plus password only. If asked how you would add it: an OAuth route that ends by issuing the same cookie; account linking only to verified emails. (SafetyWatch has Google OAuth if you want to talk about the flow.)

What is missing? Email verification and password reset (no email sending).

4.2 Web security

CSRF. Two layers in security.ts: the cookie is `SameSite=Lax`, and `originGuard` rejects any state-changing request (not GET/HEAD/OPTIONS) whose `Origin` is not this site or a configured client origin, or whose `Sec-Fetch-Site` header says `cross-site`. A request without `Origin` is allowed, because a non-browser client has no victim's cookie to abuse.

CORS. A small hand-written middleware: only configured origins get `Access-Control-Allow-Origin` with credentials, with `Vary: Origin`. Same-origin requests do not need CORS at all.

Content-Security-Policy. Strict: `script-src 'self'` with no inline scripts and no `eval`, `object-src 'none'`, `frame-ancestors 'none'` (no clickjacking), `base-uri 'self'`, `form-action 'self'`, `connect-src` only to self and the site's WebSocket. Consequences you can mention:

- The theme bootstrap (to avoid a flash of the wrong theme) is an external file, theme-init.js, not an inline script.

- Vite is told never to inline assets as `data:` URLs.
- Zod runs with `jitless: true`, because its feature probe uses `eval` and is reported as a CSP violation.
- `style-src` allows `'unsafe-inline'` only because CodeMirror injects its theme as a `<style>` element.

Other headers. HSTS when served over HTTPS, `Cross-Origin-Opener-Policy` and `Cross-Origin-Resource-Policy: same-origin`, `Referrer-Policy: strict-origin-when-cross-origin`, and a `Permissions-Policy` that allows camera, microphone and screen capture only for this site and disables geolocation, payment and USB. API responses are `Cache-Control: no-store`.

Authorisation pattern. Every resource is loaded *for a member*: `loadForParty`, `loadContractForParty`, `loadSessionForParticipant`. Someone who is not a party gets the same 404 as a missing id, so ids cannot be probed. Identity is always `req.user` or `socket.data.user`, never a field in the body.

Mass assignment. Profile updates are parsed by a **strict** Zod schema, so only whitelisted fields can be present; ratings, role and status can never be set by a member. 1.0 allowed exactly that.

Injection. Search text goes into a MongoDB regular expression, so it passes through `escapeRegExp` first and is matched literally (no ReDoS, no pattern tricks). Mongoose casts query values to schema types, which blocks operator injection like `{ "$gt": "" }` in fields that expect strings.

Uploads. Multer writes to disk under a random 32-hex-character name plus an extension from an **allow-list** that deliberately excludes HTML and SVG (both can carry script). The original name is kept only as metadata. Files are served only to the session's participants, as downloads (`Content-Disposition: attachment`), never inline. The public `/uploads` folder of 1.0 is gone.

Error handling. One format: `{ error: { code, message, fields?, requestId } }`. Services throw `AppError` helpers; anything else becomes a generic 500 with the details logged against the request id, never sent to the client. 1.0 leaked internal messages.

4.3 The data schema (MongoDB with Mongoose)

Nine collections (models/):

Collection	Key fields	Design notes
`users`	`name`, `username` (unique, lowercase), `email` (unique, lowercase), `password` (`select: false`), `headline`, `bio`, `skillsTeaching[] {skill, proficiency, yearsExperience, description}`, `skillsLearning[] {skill, targetProficiency, priority, notes}`, `availability {days[], timeSlots[], hoursPerWeek}`, `preferences {format[], sessionLength, languages[]}`, `stats {teachingRating, teachingReviews, learningRating, learningReviews, sessionsCompleted}`, `role`, `status`, `isDemo`, `onboardingCompleted`, `tokenVersion`, `lastActive`	Skills and availability are **embedded**: always read with the profile, bounded in size. `stats` is a cache recomputed from reviews and sessions. Indexes on `skillsTeaching.skill`, `skillsLearning.skill`, and `(status, onboardingCompleted, stats.teachingRating)` for discover.

Collection	Key fields	Design notes
`skills`	`name` (unique), `category`	Curated suggestion catalogue; members can still type skills not listed.
`exchangerequests`	`sender`, `receiver`, `teachingSkill`, `learningSkill`, `message`, `proposedSessions`, `proposedDurationMinutes`, `learningGoals[]`, `status`, `counterOffer {…, counteredBy}`, `statusHistory[]`, `createdContract`, `openPairKey`	**Sparse unique index on `openPairKey`** = one open exchange per pair. Indexes `(sender, updatedAt)` and `(receiver, updatedAt)` for the inbox.
`learningcontracts`	`exchangeRequest`, `userA`, `userB`, `skillA`, `skillB`, `totalSessions`, `completedSessions`, `sessionDurationMinutes`, `schedule`, `goals[] {title, completed, completedAt, completedBy}`, `status`, `endedAt`	userA teaches skillA, userB teaches skillB.
`sessions`	`contract`, `participants[2]`, `teacher`, `learner`, `skillTaught`, scheduled and actual times, `status`, `roomCode` (unique), `notes`, `codeEditorState {language, code}`, `collabState` (binary Yjs state, `select: false`), `sharedFiles[]`, `joinedBy[]`, `reviewedBy[]`, `endedBy`	`joinedBy` decides whether a session counts. `reviewedBy` is the per-member review slot. Index `(participants, scheduledStartTime)`.
`messages`	`sessionId` (the room code), `sender`, `text`	Room chat. Index `(sessionId, createdAt)`.
`reviews`	`session`, `contract`, `reviewer`, `reviewee`, `role` (TEACHER/STUDENT), `skill`, `rating 1–5`, `feedback`	**Partial unique index on `(session, reviewer)`**: one review per member per session, enforced by the database. Partial because reviews imported from 1.0 may lack a session.
`reports`	`reporter`, `reportedUser`, `reason`, `details`, `status`, `resolutionNotes`, `actionTaken`, `resolvedBy`, `resolvedAt`	Moderation queue.
`notifications`	`recipient`, `sender`, `type`, `title`, `message`, `link`, `read`	Index `(recipient, read, createdAt)` for the unread badge and list.

Why MongoDB? Honest answer: 1.0 was a MERN app, and member profiles (nested skills, availability and preferences) fit documents well. The parts that are really relational (exchange → contract → sessions → reviews) needed care: unique and partial indexes, conditional updates and recomputation instead of counters. If I started again I would seriously consider Postgres for those relationships and transactions.

Embed or reference? Embed what is owned and bounded and read together (skills in a user, goals in a contract, shared-file metadata in a session). Reference what has its own life or grows without bound (sessions, messages, reviews, notifications).

Why `statusHistory` on an exchange? An audit trail of who did what and when, shown in the UI as the negotiation history.

What is `.lean()` and the gotcha you hit with it? `.lean()` returns plain objects instead of Mongoose documents: faster, less memory, used for read-only queries. The gotcha: a binary field comes back as a BSON `Binary` (whose `length` is a method) instead of a Node `Buffer`. `toBytes()` in rooms.ts normalises both.

4.4 Correctness under concurrency: the business rules

How do you make sure two people have only one open exchange? `openPairKey` is the two user ids sorted and joined (`pairKey(a, b)`), set while the exchange is open (pending, countered or accepted with an active contract) and removed when it closes. A **sparse unique index** on it means a second open exchange for the same pair fails with a duplicate-key error, even if both people click "propose" at the same instant. A check-then-insert in code could not guarantee that.

Who can accept? `awaitingUserId` is whoever did not make the latest offer: the receiver for the original proposal, the other person after a counter. Only they can accept, decline or counter; only the person who made the latest offer can withdraw it.

What stops two conflicting responses (say, accept and withdraw at the same moment)? Every transition is a `findOneAndUpdate` whose filter repeats the state it validated: `{ _id, status: <status I read>, updatedAt: <updatedAt I read> }`. If anything changed in between, nothing matches, and the user gets "This exchange was just updated. Refresh and try again." That is **optimistic concurrency** (compare-and-set), with no locks.

Accepting creates a contract. What if that insert fails after the status changed? There is no multi-document transaction, so the code **compensates**: it puts the exchange back to its previous status and pops the history entry, then rethrows. Why not a transaction? MongoDB transactions need a replica set, and the zero-config development mode runs a standalone in-memory server. Compensation works everywhere. On Atlas (a replica set) a transaction would be the cleaner option.

How is scheduling limited to the agreed number of sessions? `completedSessions + sessions still scheduled or active` must be below `totalSessions`. (Honest note: that count-then-insert is not atomic, so two simultaneous "schedule" clicks could overbook by one. Fix: a conditional increment of a "booked" counter on the contract.)

When does a session count as completed? Joining the room adds you to `joinedBy` with `$addToSet` and flips the session from SCHEDULED to ACTIVE with a conditional update. Ending it checks that **every participant is in `joinedBy`**: if so it becomes COMPLETED, otherwise CANCELLED. So nobody can farm completed sessions or reviews alone. The end itself is `findOneAndUpdate({ _id, status: 'ACTIVE' })`, so it happens once even if both click "end".

How do you guarantee one review per person per session? Two layers. First, claim the slot: `updateOne({ _id, reviewedBy: { $ne: me } }, { $addToSet: { reviewedBy: me } })`; if nothing was modified, you already reviewed. Then insert the review; the partial unique index `(session, reviewer)` is the database-level backstop. If the insert fails for another reason, the slot is released.

How are ratings computed? `recomputeUserStats` runs an aggregation over all reviews of that member, grouped by role (as teacher, as learner), and counts completed sessions, then writes the `stats` object. **Recomputed, never incremented**, so retries and races cannot drift the numbers. Ratings per skill are another aggregation. New members show "No reviews yet": `null`, not a made-up 4.9. 1.0 invented default ratings, streaks and "mastery" bars; all removed.

4.5 The matching algorithm

matching.ts scores how well *they* fit *me*, out of 100, in five parts:

Part	Full marks	Partial marks
Skill fit (50)	two-way: they teach something I want *and* I teach something they want	30 if only they teach what I want; 20 if only I teach what they want
Availability (20)	at least one shared day *and* one shared part of the day	12 for shared days only; 8 for shared time slots only
Level (10)	their proficiency in the skill is at or above my target level	6 if one level below; 3 if further below
Language (10)	at least one shared language	0
Preferences (10)	6 for a shared format (video, chat, …) + 4 for the same session length	

Skill names are compared case-insensitively. Each part that scores adds a human-readable reason, and the response includes the breakdown so the UI can show "what this score is made of". Nothing in common scores zero; 1.0 had a minimum score floor, which made unrelated people look like matches.

How does Discover rank and paginate? users.ts: MongoDB filters first (active, onboarded, not me, optional text search, proficiency, day, minimum rating, online now), takes at most 500 candidates, then scores each against me in Node, sorts by score with rating as a tie-break, and **only then** slices out the requested page. 1.0 paginated first and ranked within the page, so the best match could be on page 3. Visitors who are not signed in get rating order, since there is nobody to match against.

Why not compute the score in the database? It is a function of two users' nested arrays, so it cannot be indexed. Computing it in Node over a bounded candidate set is simple and fast enough at this size. At scale: pre-filter candidates by skill with the multikey indexes (only people who teach something I want or want something I teach), precompute top matches in a background job when profiles change, or move to a search engine with custom scoring.

4.6 The learning room: WebRTC calls

useWebRTC.ts is the most likely deep-dive.

What is WebRTC, briefly? A browser API for sending audio, video and data directly between two browsers. The browsers need to exchange two things through some other channel first (**signalling**): a session description (SDP: codecs, media) as an offer and an answer, and **ICE candidates** (possible network addresses). Here Socket.IO is the signalling channel; the server only relays those messages within the room and never sees media.

What are STUN and TURN? STUN tells a browser its public address as seen from outside its NAT, so two browsers can try to connect directly. Default here: Google's public STUN server. When both sides are behind strict NATs or firewalls, a direct path is impossible and a **TURN** server must relay the media. `ICE_SERVERS` accepts a TURN server; the demo has none, which is a known limitation.

What is "perfect negotiation" and why use it? A pattern (from the WebRTC spec authors) where either side can start negotiating at any time. If both send offers at once (a "glare" collision), one side is **polite** and rolls back to accept the other's offer, the other is **impolite** and ignores the incoming one. Roles come from the server: `polite = myUserId < partnerId`, a stable tie-break that both sides compute differently. Benefit: no "caller" and "callee", and renegotiation for screen sharing or a camera that appears late "just works".

Walk me through a call.

1. On entering the room, open camera and microphone (`getUserMedia`, 720p ideal, echo cancellation and noise suppression). If the camera fails, retry audio only. If both fail, explain why in words (blocked permission, no device, device in use) and keep the other tools usable.
2. When both participants are present, create a **fresh** `RTCPeerConnection` for that "epoch". `peerEpoch` increases every time the partner (re)joins, so a reload or reconnect gets a clean connection instead of reusing a broken one.
3. Add local tracks. If there is nothing to send, add receive-only transceivers so you still see and hear the other side.
4. `onnegotiationneeded` → `setLocalDescription()` (creates the offer) → send it. Incoming offers are handled by the perfect-negotiation logic and answered. ICE candidates are trickled to the other side as they are found.
5. **Signals that arrive before the connection exists are queued** and replayed once it is created. 1.0 dropped early ICE candidates, so calls sometimes never connected.
6. `ontrack` sets the remote stream (a new `MediaStream` object so the `<video>` element notices added or replaced tracks).
7. If the connection state becomes `failed`, call `restartIce()` to try a fresh network path.

How does screen sharing work without dropping the call? `getDisplayMedia`, then `RTCRtpSender.replaceTrack(screenTrack)` on the existing video sender. Swapping a track on the same sender needs no renegotiation. Stopping (or the browser's own "Stop sharing" button, which ends the track) swaps the camera back.

Mute and camera off? Toggle `track.enabled`, which sends silence or black frames without renegotiating, and tell the other side through a `media:state` socket event so their UI can say "camera off" instead of showing a frozen frame.

What if someone opens the room in a second tab? The server keeps one seat per member per room. The newer tab takes the seat; the old tab gets `room:replaced` and shows "This session is open in another tab" with a "Use this tab instead" button that takes the seat back.

How was the call tested? Playwright launches Chromium with `--use-fake-device-for-media-stream` and `--use-fake-ui-for-media-stream`, so two real browser contexts get generated camera streams and auto-accepted permissions, connect, and the test checks video is flowing.

4.7 The learning room: shared code, notes and whiteboard (Yjs)

What is a CRDT and why use one? A Conflict-free Replicated Data Type: a data structure that several copies can edit independently and that always merges to the same result, whatever order the changes arrive in. Yjs is a CRDT library for text, arrays and maps. Two people typing in the same line at the same time, or one typing offline, never lose an edit. The alternative, operational transformation (Google Docs), needs a central server to transform operations; with a CRDT the server can simply merge.

What is in the shared document? One `Y.Doc` per session (rooms.ts): a `Y.Text` for code, a `Y.Text` for notes, a `Y.Map` for metadata (the editor language), and a `Y.Array` of strokes, each a `Y.Map` with an id, author, tool, colour, size and a `Y.Array` of points.

How do updates flow?

1. Client: the doc emits an `update` (a small binary diff) for every local change. Unless the change came from the network (tagged with origin `REMOTE`, to avoid echoing it back), it is sent as a `doc:update` socket event (useRoom.ts).
2. Server: checks the sender holds a seat in the room, the token bucket allows it, and the update is a non-empty binary under 256 KB; applies it to the server's copy of the doc (a malformed update throws and is rejected); marks the room dirty; relays the update to the other participant.
3. Persistence: 2 seconds after the last change (debounced), the server encodes the whole doc and saves it to `sessions.collabState`, plus plain-text snapshots of code and notes, so a session's content is readable without decoding Yjs (sessions from 1.0 only had plain text, and `buildDoc` can start a doc from it). Capped at 2 MB. When the last member leaves, or the session ends, or the process gets `SIGTERM`, it saves immediately and unloads the room.

What does a late joiner or a reconnecting client get? On `room:join` the server sends the full encoded state. The client applies it, then computes what *it* has that the server lacks, using the server's **state vector** (`Y.encodeStateAsUpdate(doc, stateVectorFromServer)`), and sends that. That is how edits typed while the connection was down are not lost.

How is the doc loaded on the server under concurrent joins? `RoomStore.open` keeps a map of in-flight loads, so two people joining at the same moment share one database read and one `Y.Doc`, instead of each creating their own and one overwriting the other.

How is each editor bound to the doc?

- **Code:** CodeMirror 6 with `y-codemirror.next`'s `yCollab(text, …)` and a `Y.UndoManager`, so undo only undoes your own changes. 1.0 used Monaco from a CDN; CodeMirror is smaller, works on touch devices and needs no third-party script (which the CSP would forbid anyway).
- **Notes:** a plain `<textarea>`. On each change, textSync.ts computes the smallest single replacement between the old and new text (common prefix and suffix) and applies only that to the `Y.Text`. For remote changes, `transformPosition` moves your caret by the remote delta so it does not jump. Both functions have unit tests.
- **Whiteboard:** an HTML canvas with pointer events (mouse, touch and pen, with pointer capture). Coordinates are normalised to a fixed board size so drawings line up whatever the screen size (1.0 drew offset on resized screens). A stroke is created in one transaction; points are batched (every 4 points) rather than one message per pixel. Repainting is scheduled with `requestAnimationFrame` from `observeDeep`. Undo removes your own last stroke only. Colours follow the theme.

4.8 Running untrusted code: the sandbox

The threat. The code in the shared editor may have been typed by the other participant. If it ran in the app's own page, it could read the page, call the API with your cookie, and act as you.

The design (runSandbox.ts, runner.html):

1. A hidden `<iframe sandbox="allow-scripts">` loads `/sandbox/runner.html`. Without `allow-same-origin`, the frame has an **opaque origin**: no cookies, no storage, no access to the parent's DOM, and requests it makes are cross-origin without credentials.
2. The server serves that one page with its own CSP: `default-src 'none'` (so `fetch`, images and sockets are all blocked: **no network**), scripts only inline/eval/blob for the runner itself, workers only from `blob:`, and `frame-ancestors 'self'`.
3. Inside, the code runs in a **Web Worker** created from a blob URL, with `console` replaced so output is posted back. An infinite loop blocks only the worker.
4. Communication is `postMessage`, and the parent checks `event.source === iframe.contentWindow` so other frames cannot inject output.
5. After **5 seconds** the parent removes the iframe, which kills the worker even mid-loop. Output is capped at 200 lines of 2000 characters and sanitised, because the producer is untrusted.
6. Only JavaScript runs. 1.0 had a "multi-language runner" that only pretended; it was removed rather than faked.

Why `(0, eval)(code)` in the worker? An *indirect* eval runs in global scope, so the user's code cannot see the runner's local variables.

4.9 Socket.IO: realtime outside the room

Authentication. During the handshake the server reads the session cookie from the headers and resolves the user exactly like an HTTP request; suspended or invalid → refused. CORS does not apply to WebSocket upgrades, so `allowRequest` checks the `Origin` header itself.

Rooms. Each socket joins `user:<id>` (for notifications and forced disconnects). A learning-room seat joins `room:<code>`, but only after the server checks the user is one of the session's two participants and the session is not closed. Non-participants get the same "does not exist" as a bad code.

Notifications. Stored in MongoDB and pushed live to `user:<id>` as `notification:new`; the client shows a toast and updates the unread badge.

Presence. A map from user id to socket ids; the first socket announces "online", the last disconnect announces "offline". (Weakness to own: it is broadcast to every connected user, not just people you have a relationship with. It powers the "online now" filter on Discover. A stricter design would only tell contacts.)

Abuse limits. Token buckets per socket: chat 8 per 4 s, document updates 120/s, call signals 200/s. Every payload is validated with a shared Zod schema before use; invalid ones are dropped.

4.10 Frontend

Stack. React 19, Vite 8, Tailwind CSS 4, React Router 7, TanStack Query, react-hook-form, CodeMirror 6.

Why TanStack Query? It is a cache for server state: loading and error states, deduplication, background refetching, and **invalidation** after mutations (`invalidateExchangeData(queryClient)` after any exchange action refreshes every list that depends on it). The current user is just another query (`['me']`). That replaced a lot of hand-written `useEffect` fetching in 1.0.

Forms. react-hook-form with `zodResolver(sharedSchema)`. Server field errors (`error.fields`) are mapped back onto the fields with `applyServerErrors`.

Design system. components/ui: buttons, fields, native `<dialog>` modals (mounted only while open, focus restored on close), tabs, menus, toasts, loading/empty/error states, a confirm dialog for destructive actions. Colours are **semantic tokens** (`bg-surface`, `text-muted`, `bg-teach-bg`) defined for light and dark themes; teal always means "teaches" and amber "wants to learn".

Accessibility (WCAG 2.2 AA). A unit test (tokens.test.ts) computes the contrast ratio of every foreground/background token pairing in both themes. ESLint's `jsx-a11y` rules. Every e2e page check runs axe-core at desktop and phone sizes. Labels on every control, ordered headings, touch targets of 36–44 px, no meaning by colour alone.

Performance. Pages behind sign-in are code-split (`React.lazy`). The build writes brotli and gzip copies of hashed assets; the server sends those with a one-year `immutable` cache, and injects a preload for the UI font. Fonts are self-hosted (1.0 hot-linked Google Fonts and stock photos).

Routing and SEO in an SPA. web.ts knows the app's routes, so an unknown URL gets a **real 404 status** with the app shell, not a 200. For public profiles it fills in the page title, description and Open Graph tags on the server (escaped), so link previews work. It also serves `robots.txt` and a sitemap.

4.11 Testing and CI

What is tested? 224 unit and integration tests:

- **Server (123):** the real Express app and Socket.IO server against an in-memory MongoDB (`mongodb-memory-server`), with Supertest for HTTP and real socket clients. Auth, exchanges, sessions, users, notifications, admin, realtime, the web server, env validation, the demo seeding and the 1.0 → 2.0 migration on 1.0-shaped data. Includes security regression tests (a non-party gets 404, cannot set role, etc.).
- **Client (88):** UI components, library functions, the text-sync algorithm, the colour-contrast test.
- **Shared (13):** the Zod schemas.

End to end (Playwright): two real browsers negotiate an exchange, hold a WebRTC call with fake camera streams, edit code and notes together, end the session and review each other. Every page at desktop and phone size is checked for horizontal overflow, console errors and axe violations. The sandbox is tested against hostile code: code that tries to `fetch('/api/auth/me')` with your session, read storage, or loop forever.

Why an in-memory MongoDB rather than mocks? The rules that matter live in queries, unique indexes and conditional updates; mocks would test my assumptions about MongoDB, not MongoDB.

CI (.github/workflows/ci.yml): lint (zero warnings), Prettier check, typecheck, tests, build; then a second job runs the e2e suite against the build and uploads the Playwright report on failure. A Husky pre-commit hook runs ESLint and Prettier on staged files.

4.12 Deployment and operations

How is it deployed? A multi-stage **Dockerfile**: build stage installs everything and builds client and server (the server is bundled into one file); runtime stage on `node:24-slim` installs only the server's production dependencies, copies the built output, runs as the unprivileged `node` user, and has a `HEALTHCHECK` on `/api/health/ready`. **Render** runs it from render.yaml (free plan, Singapore, custom domain `skillswap.arvin.life`, auto-deploy on commit, `JWT_SECRET` generated by Render). Database: **MongoDB Atlas**.

Health checks. `/api/health` (process alive, liveness) and `/api/health/ready` (database reachable, readiness). The difference: a load balancer should stop sending traffic to an instance whose database is down (readiness), but only restart it if the process itself is stuck (liveness).

Graceful shutdown. On `SIGTERM` the server saves every open shared document, closes sockets and the HTTP server, then exits. Container platforms send `SIGTERM` before killing.

Logging. pino structured JSON logs with a request id per request (also returned as `X-Request-Id`), so a user's error message can be matched to the log line.

The public demo. `DEMO_MODE`, `SEED_DEMO_DATA` and `DEMO_RESET_HOURS=24`: the empty database is seeded at startup and wiped and re-seeded daily, so vandalism does not last.

Free-plan caveats. The instance sleeps when idle (first visit is slow), and there is no persistent disk, so shared files are lost on restart.

Zero-config development. `npm install && npm run dev` with no `.env` starts an in-memory MongoDB, seeds demo data and turns on demo sign-in.

Migration from 1.0. `npm run migrate` upgrades a 1.x database in place, idempotently: cleans duplicates *before* building new unique indexes, normalises old shapes, closes duplicate open exchanges for the same pair, recomputes every member's stats, and flags non-demo accounts that still use the leaked 1.0 seed password.

---

5. Behavioural questions and stories

Situation, action, result, lesson. Under two minutes each. Reword to match what happened.

"Tell me about a project you improved significantly."

Rebuilding SkillSwap 1.0 into 2.0.

- **Situation.** 1.0 worked as a demo but had a hard-coded fallback JWT secret (anyone could forge a login), the token in `localStorage` (stealable by any XSS), unauthenticated skill creation, a public uploads folder, ratings and "mastery" bars that were invented, a code runner that pretended to support many languages, and no tests.
- **Action.** Audited it, wrote the list (it is the changelog), then rebuilt in layers: a shared schema contract, services that own every rule, conditional updates, httpOnly cookie sessions with token versions, a real sandbox, a design system with accessibility checks, and tests for each fix.

- **Result.** 224 unit/integration tests, end-to-end tests including a real two-browser call, CI on every push, and every number on screen computed by the server.
- **Lesson.** Removing fake features is part of shipping. A "4.9 stars" that nobody earned is a bug.

"Tell me about a hard bug."

Pick one you can describe in detail:

- **Calls that never connected.** ICE candidates arrived before the `RTCPeerConnection` existed and were dropped. Fix: one long-lived signal listener that queues until a connection exists, a fresh connection per "both present" epoch, and perfect negotiation so either side can (re)start.
- **Concurrent joins clobbering the shared document.** Two people joining at once each loaded the saved state into their own `Y.Doc`. Fix: cache the in-flight load promise per room so both share one doc.
- **Binary data from lean queries.** The saved Yjs state came back as a BSON `Binary` from `.lean()`, so applying it failed. Fix: `toBytes()` that normalises both shapes, with a test.

"Tell me about a trade-off."

- **JWT with a token version vs a session table.** Sign-out-everywhere for one integer, at the cost of per-device revocation.
- **Compensation vs transactions.** Works on a standalone MongoDB (zero-config dev), at the cost of a brief window where an exchange shows accepted without a contract.
- **Ranking in Node over 500 candidates.** Correct ranking and explainable scores now; needs pre-filtering or precomputation at scale.
- **In-memory room state.** Simple and fast; limits it to one instance.

"Tell me about security work you did."

The code sandbox (4.8) or the auth fixes (4.1). For the sandbox, explain the threat in one sentence ("my partner's code runs in my browser"), then the four layers: opaque origin, no-network CSP, worker, timeout.

"How do you ensure quality?"

The shared contract (one schema, both sides), one serializer for responses, conditional updates for anything that must happen once, recomputation instead of counters, tests against a real (in-memory) MongoDB, e2e tests in real browsers with accessibility checks, CI that blocks on lint, types, tests and build. "A bug fix comes with a regression test" is a written rule in the repo.

"What would you do differently?"

- Consider Postgres for the relational core and use transactions.
- Opaque session ids for per-device sign-out.
- Design for more than one instance from the start (Redis adapter for Socket.IO, shared Yjs persistence such as `y-redis`).
- A TURN server from day one; calls failing on corporate networks is the most visible failure a user can hit.

"What are you most proud of?"

The learning room working end to end in an automated test: two real browsers, a real WebRTC call, live shared editing, completed session, reviews. It proves the hardest part of the product works on every push.

---

6. The awkward questions

"Logout doesn't revoke the JWT, does it?" Correct: it clears the cookie in that browser. A copied token stays valid until it expires (7 days). Password change or suspension revokes all tokens via the token version. Per-device revocation needs a server-side session id; that is the next step.

"Anyone can see who is online?" Yes, presence is broadcast to all signed-in users. It is a product choice for the "online now" filter, but a privacy-minded version would limit it to contacts or let people hide it.

"There's a `SESSION_EARLY_JOIN_MINUTES = 15` constant. Is it enforced?" No: it is defined in `shared/src/constants.ts` but not used, so a room can be opened at any time. Either enforce it in `room:join` or delete it. Better to know before they find it.

"Can two quick clicks book more sessions than the contract allows?" Possibly by one: the count-then-insert is not atomic. Fix: a conditional `$inc` of a booked counter with a `$lt: totalSessions` filter.

"Calls fail on my office network." Without a TURN server, two strict NATs cannot connect directly. Add one to `ICE_SERVERS` (coturn, or a managed TURN service).

"Uploaded files disappear." On the free Render plan there is no persistent disk. In production `UPLOAD_DIR` must be a volume, or files go to object storage (S3/R2) with signed download URLs.

"Why is `CLAUDE.md` in your repo? Did an AI write this?" Answer honestly about the tools you used and what you decided yourself. The file is a conventions guide for contributors and coding agents. Then demonstrate ownership: pick any part (perfect negotiation, the sandbox, the token version) and explain it without notes.

"Why MongoDB for clearly relational data?" It was the 1.0 stack and profiles fit documents. The relational parts are protected with unique and partial indexes, conditional updates and recomputation. Starting fresh, I would weigh Postgres seriously.

"Search is a regex. Does that scale?" A case-insensitive unanchored regex cannot use an index, so it scans the candidate set. Fine for thousands of members; at scale, a MongoDB Atlas Search / text index or an external search engine.

"The README says 200+ tests, the changelog says 224." Both true; run `npm test` and quote the exact number you see (224 today: 13 + 123 + 88).

---

7. System design follow-ups

"Run several instances."

1. Socket.IO Redis adapter so events reach sockets on other instances.
2. Sticky sessions or WebSocket-only transport, because Socket.IO's polling fallback needs requests to hit the same instance.
3. The Yjs room must live in one place: route each room to one instance (consistent hashing on the room code), or use a shared Yjs backend (`y-redis`, Hocuspocus) that merges updates across instances.
4. Presence and rate-limit counters move to Redis.
5. Uploads to object storage.

"10,000 concurrent learning rooms." Media is peer to peer, so the server only carries signalling and small Yjs updates: cheap. TURN relays become the bandwidth cost for users behind strict NATs. The in-memory docs (bounded at 2 MB each) and the debounced saves are the server's load; shard rooms across instances.

"Group sessions (one teacher, ten learners)." Peer-to-peer mesh breaks down beyond about four participants (each uploads a stream to everyone). Switch to an SFU (Selective Forwarding Unit) such as mediasoup or LiveKit: each client uploads once and the server forwards.

"Better matching." Learn from outcomes: which matches led to accepted exchanges and completed sessions. Use that to tune the weights (still explainable), or add collaborative filtering. Precompute top matches per user in a background job.

"Scheduling across time zones." Store availability as UTC intervals per weekday derived from the member's IANA time zone (there is a `timezone` field), and compute overlap in UTC.

"Notifications by email." A job queue (BullMQ on Redis) fed by the same `notify()` function, with per-user preferences and digest batching.

---

8. Concepts to explain from scratch

Concept	Plain explanation
JWT	A signed token (header.payload.signature). Anyone can read it; only the secret holder can create a valid one.
HS256	HMAC with SHA-256: one shared secret both signs and verifies.
Token version	A counter on the user copied into each token; bump it to invalidate every token issued before.
bcrypt / salt / cost	Slow salted password hash; the cost is a power of two of rounds; reads at most 72 bytes.
`__Host-` cookie prefix	Browser enforces Secure, Path=/ and no Domain, binding the cookie to one host.
CSRF / Origin / Sec-Fetch-Site	Forged cross-site requests; blocked by SameSite cookies plus checking where the request came from.
CSP	A header listing where scripts, styles, connections and frames may come from; blocks injected scripts.

Concept	Plain explanation
Sandboxed iframe / opaque origin	A frame with no origin: no cookies, storage or access to the parent.
Web Worker	JavaScript on a background thread; cannot touch the DOM; can be terminated.
WebRTC	Browser-to-browser audio/video/data.
SDP offer/answer	Each side's description of its media and codecs, exchanged during signalling.
ICE / STUN / TURN	Finding a network path; discovering your public address; relaying when no direct path exists.
Perfect negotiation	Symmetric negotiation where a fixed polite side yields on collisions.
`replaceTrack`	Swap the media on a sender without renegotiating.
CRDT / Yjs	Data structures whose copies merge to the same result regardless of order.
State vector	A summary of which changes a Yjs doc has seen; used to compute exactly what the other side lacks.
Debounce	Wait until changes stop for a while before acting (saving here).
Token bucket	A rate limiter that refills continuously and allows short bursts.
Optimistic concurrency	Update only if the record still has the version you read.
Compensating action	Undoing a completed step when a later step fails, instead of a transaction.
Sparse / partial unique index	A unique index that only covers documents with the field (sparse) or matching a filter (partial).
Aggregation pipeline	MongoDB's server-side data processing: match, group, project.
`.lean()`	Return plain objects instead of Mongoose documents.
TanStack Query invalidation	Mark cached server data stale so it is refetched.
WCAG 2.2 AA / contrast ratio	Accessibility standard; normal text needs at least 4.5:1 contrast.
Liveness vs readiness	"Is the process alive?" vs "can it serve traffic right now?"
Multi-stage Docker build	Build with all tools, ship only the runtime output.

---

9. Before the interview

Fix

- [ ] Add SkillSwap to the resume, and correct the SynChat line (its JWT/MongoDB description belongs here).
- [ ] Decide whether to enforce or delete `SESSION_EARLY_JOIN_MINUTES`.
- [ ] Optional quick wins that are great to talk about: per-device sign-out (session id in the JWT + a sessions collection), presence limited to contacts, atomic session booking.

Run once, so you are quoting what you saw

- [ ] `npm test` (224 today) and `npm run build && npm run e2e`.
- [ ] `npm run dev`, sign in as `arjun` in one browser and `rahul` in a private window, open room `SWAP-DEMO-ROOM`: call, share screen, type in the editor, draw, run `while(true){}` in the sandbox and watch it time out, run `fetch('https://example.com')` and watch it fail.
- [ ] Open the live demo at skillswap.arvin.life the day before (the free instance sleeps).

Five-minute demo script

1. Discover: open a match score's breakdown.
2. Propose an exchange; in the other browser counter it; accept in the first.
3. Schedule a session from the contract.
4. Both join the room: video, screen share, code edit live, run code, whiteboard.
5. End the session and leave a review; show the rating update.

Read these files until you can explain every line

1. server/src/services/exchanges.ts: `createExchange`, `actOnExchange`
2. server/src/services/sessions.ts: `endSession`, `reviewSession`
3. server/src/services/matching.ts
4. server/src/realtime/index.ts and rooms.ts
5. client/src/room/useWebRTC.ts and useRoom.ts
6. client/src/room/runSandbox.ts and runner.html
7. server/src/lib/session-token.ts, middleware/auth.ts, middleware/security.ts

Questions to ask them

- How do you test realtime features (calls, collaborative editing) in CI?
- When do you choose a document database over a relational one?
- How do you approach accessibility: audits at the end, or checks in CI?